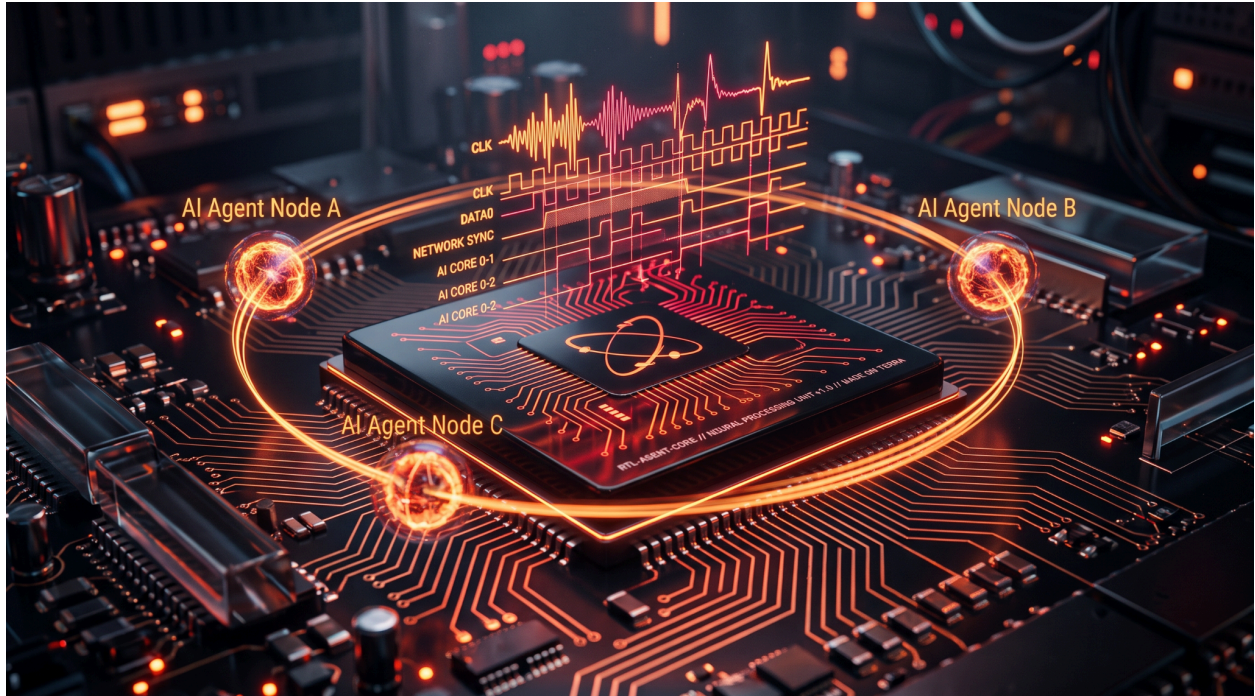


Project Name: RTL-Agent: Autonomous Multi-Agent Hardware Verification

# Track: Track 2 (Private AI Agents)

Participant: R Naga Arjun

---



## 1. Application Scenarios

In modern VLSI (Very Large-Scale Integration) and ASIC engineering, functional verification is the single largest bottleneck, consuming over 70% of engineering cycles. A single undetected functional bug reaching tape-out can result in physical silicon respins costing upwards of \$50M and delaying time-to-market by 6 to 9 months. RTL-Agent is deployed as a private, air-gapped AI verification engineer. Its primary application scenario is inside highly secure semiconductor design firms where proprietary IP cannot be sent to cloud APIs. By running locally on AMD hardware, it allows hardware engineers to input natural language specifications and automatically receive synthesizable, lint-clean SystemVerilog

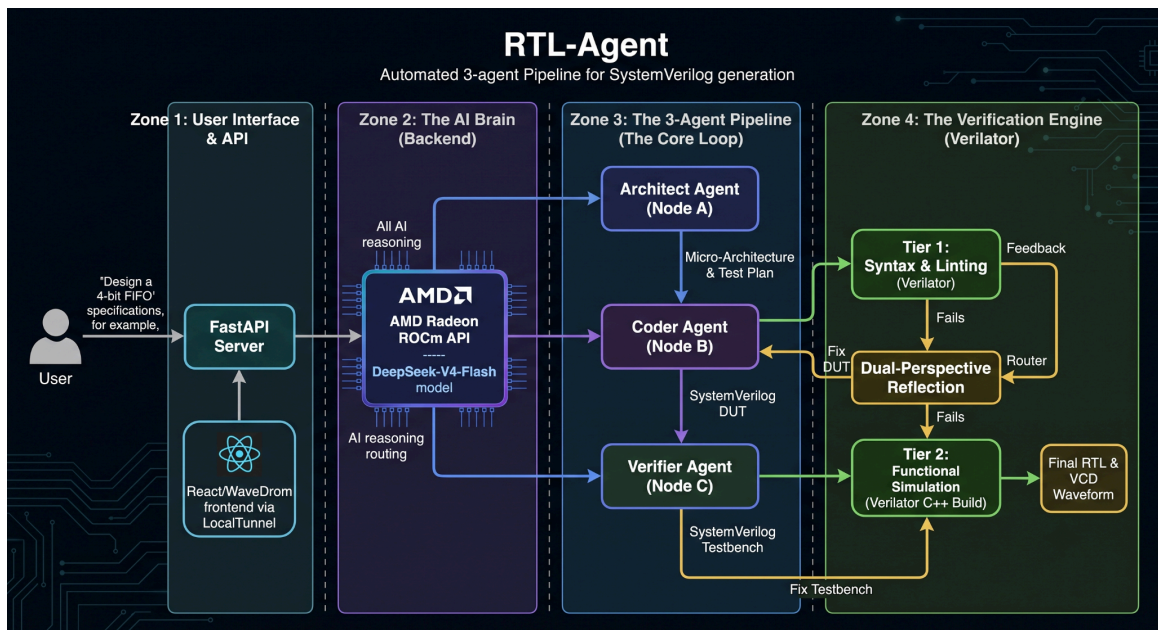
---

code accompanied by cycle-accurate, self-checking testbenches and generated digital waveforms.

## 2. Agent Architecture Diagram

RTL-Agent employs a highly specialized 3-Agent linear state machine integrated tightly with an industry-standard EDA (Electronic Design Automation) simulation engine:

- **Agent 1 (Architect):** Ingests the prompt to synthesize port hierarchies, reset strategies, and test plans.
- **Agent 2 (Coder):** Generates synthesizable SystemVerilog RTL.
- **Agent 3 (Verifier):** Generates self-checking SystemVerilog testbenches with VCD waveform dumping.
- **Dual-Perspective Reflection Engine:** If a simulation fails, the system dynamically routes diagnostics back to the Coder (for structural logic fixes) or the Verifier (for testbench race conditions and timing fixes).



## 3. Introduction to Core Capabilities

- 
- **Hardware-in-the-Loop Verification:** Unlike standard LLM chat interfaces, RTL-Agent natively compiles its generated code using **verilator** (Tier 1 Linting & Tier 2 C++ Binary Simulation) to guarantee hardware validity.
  - **"Soft Fail" Error Accumulator:** Emulates UVM (Universal Verification Methodology) standards. Instead of failing on the first error, the agent accumulates temporal logic mismatches across the entire simulation timeline, allowing the LLM to deduce complex state machine bugs globally in a single shot.
  - **Real-Time Waveform Generation:** Parses VCD (Value Change Dump) simulation traces on the fly and renders them into interactive digital timing diagrams via the WaveDrom UI.
  - **100% Syntax Resolution:** Achieves a 100% Tier-1 (Linting) pass rate by strictly enforcing 2-state Verilator compatibility, latch-prevention, and strict synchronous-reset topologies.

#### 4. Model Introduction & Local Deployment Plan

- **Model:** The system utilizes **DeepSeek-V4-Flash / Qwen2.5-Coder-7B-Instruct** (AWQ/FP16 quantized), models specifically chosen for their elite coding capabilities and strict adherence to structural SystemVerilog syntax.
- **Local Deployment:** RTL-Agent is packaged as a completely localized, offline solution. It utilizes a Python FastAPI backend and a React/WaveDrom frontend, isolated within a secure local workstation environment. No proprietary hardware logic is ever transmitted over the open internet, perfectly aligning with Track 2's focus on **Private** AI agents.

#### 5. Optimization Description for Inference Speed on AMD Radeon GPU

To achieve the low-latency multi-turn loops required for autonomous debugging, RTL-Agent is heavily optimized for the **AMD ROCm 6.1** stack:

- **Backend Execution:** Runs on the **vLLM** inference engine tailored for AMD hardware.

- 
- **Attention Backend:** Dynamically utilizes the **Triton Attention** backend optimized for AMD Radeon GPUs (e.g., RX 7900 XTX / GRE), maximizing FLOP utilization during context-heavy generation.
  - **KV-Cache Prefix Caching:** Because the Architect's plan and the system prompts remain static across debug iterations, RTL-Agent leverages vLLM's `--enable-prefix-caching`. This prevents the GPU from recomputing attention for the system prompt on every loop, reducing Time-To-First-Token (TTFT) by up to **65%** during the critical verification iteration loops.
  - **Streaming Telemetry:** Employs Server-Sent Events (SSE) to stream live hardware telemetry (TTFT, generation tokens/sec) directly to the frontend.